

FORGE FDE ACADEMY · SOLUTION ARCHITECTURE STUDIO

Architecture Discovery Reference Architectures Solution Decoding

The client hands you one sentence and three people who disagree about it. Ninety minutes later you owe them a scope you can defend. This session is how that is done — repeatably.

Discovery

Reference Architectures

Solution Canvas

90-Minute Decode

WHY THIS SESSION

The brief is never the problem — and that is the job



What the client actually hands you

- One sentence: “add AI to pharmacovigilance”, “put a copilot on our claims desk”.
- Three stakeholders who each believe it means something different — and are each partly right.
- A pile of systems nobody has fully mapped since the last acquisition.



What you owe them by end of day one

- A bounded scope with an explicit “not in scope” list that the loudest stakeholder has accepted.
- An architecture traceable to a reference pattern — and the places you deliberately departed from it.
- An estimate with its confidence stated, and the spikes that will collapse the uncertainty.

The four moves, in order — each one narrows the next



DISCOVER

What exists, what it talks to, what is rotting



ANCHOR

Which reference architecture is your baseline



DECODE

Loose brief → bounded, defensible scope



COMMIT

Canvas, estimate, confidence, spikes

The FDE's edge is not knowing more patterns than the client's architects. It is being willing to ask the question that makes the disagreement visible in the first hour — instead of discovering it in month four, when the code is written.

01



Architecture Discovery

Existing-system inventory · integration points · technical debt audit. You cannot architect a change to a system you have not mapped — and the map is never in the documentation.

DISCOVERY · 1 OF 3

Existing-system inventory — what is actually running, not what is documented

Rule zero: the architecture diagram on the wall is a historical artefact. Inventory from running evidence — deploy pipelines, DNS, billing, and the on-call rota — because those cannot lie about what is live.

Systems of record	What holds the truth? Which system wins in a conflict?	<i>The safety database, the CRM, the ledger. There is only ever one winner — find it.</i>
Data & residency	Where does data live, who owns it, what may leave the region?	<i>Ask for the data classification, not the schema. Residency kills more designs than latency.</i>
Identity & access	Who authenticates? Which directory is authoritative?	<i>SSO provider, service principals, break-glass accounts, the shared password in a vault.</i>
Runtime & hosting	What actually runs where — cloud, on-prem, someone's VM?	<i>Read the deploy pipeline and the cloud bill. Both are honest; the wiki is not.</i>
Vendors & contracts	What is bought, what is built, what is locked in until when?	<i>Renewal dates are architecture constraints. So is the exit clause.</i>
People & ownership	Who is on-call? Who says yes? Who quietly says no?	<i>An unowned system is a risk item, not a component.</i>

HIGHLIGHT — the two-hour inventory: ask for read access to the CI/CD pipelines, the cloud billing export, and the on-call schedule. Those three artefacts reconstruct 80% of a real architecture faster than a week of interviews — and they surface the systems nobody wanted to mention.

DISCOVERY · 2 OF 3

Integration points — every arrow on the diagram is a contract, a failure mode, and an owner



For every integration, capture six things

- Contract — the schema, the version, and whether it is versioned at all.
- Direction & trigger — push or pull, event or batch, who initiates.
- Volume & latency — peak, not average. Averages hide the failure.
- Failure behaviour — what happens when it is down? Retry, queue, or silent data loss?
- Owner — a named human who can change it, and their change-approval path.
- Auditability — is the exchange logged in a way a regulator would accept?



Integration smells that reprice a project

- “It's just a CSV drop” — an unversioned contract with no owner and no schema check.
- “We have an API” — undocumented, unversioned, rate-limited at a number nobody knows.
- “The nightly batch” — your real-time AI feature now inherits a 24-hour freshness floor.
- “IT will give us access” — an unscheduled dependency on a team with its own backlog.
- “We'll just read the database directly” — coupling to someone else's schema; breaks on their next release.



MINI CASE — the integration that ate the estimate. A team scoped an AI triage assistant at eight weeks. Discovery found the source system exposed no API — the vendor's integration module was licensed separately, procurement took eleven weeks, and the contract renewal fell mid-project. The AI work was still eight weeks. The project was five months. Nothing about the model changed; everything about the plan did. **Integration discovery is estimate discovery.**

Draw it as a context diagram, not a component diagram. One box for your system, one box per external party, one labelled arrow per contract. If an arrow has no named owner, it is not an integration — it is a risk with an optimistic label.

DISCOVERY · 3 OF 3

Technical debt audit — which debt blocks this project, and which is just untidy

The discipline: you are not auditing the estate's health — that is a consulting engagement nobody asked you for. You are answering one question: **which debt stands between this brief and production?**



BLOCKING

The project cannot ship without fixing it. Goes in the plan, with an owner and a date.

e.g. No test environment with production-like data. No audit log on the system you must write to.



TAXING

You can ship, but every increment costs more. Price it into the estimate — do not silently absorb it.

e.g. No CI. Manual releases. A schema so denormalised every query is an archaeology exercise.



LATENT

Harmless today, detonates under the new load or the new access pattern you are about to add.

e.g. A batch job that works at 10k rows; your AI feature will call it at 10M. No rate limiting.



IRRELEVANT

Real debt, genuinely not your problem. Name it, log it, hand it back. Do NOT fix it for free.

e.g. The legacy VB app in Finance. Ugly, unrelated to this brief, someone else's roadmap.

The four debts that specifically break AI projects

No ground truth

Nobody has a labelled set of correct answers — so no eval harness, so no way to prove the thing works.

No environment

No non-production place with realistic data. You cannot test an AI system on synthetic toy data and claim readiness.

No observability

You cannot see what the current process does, so you cannot show your system did it better.

No decision owner

No named human accountable for the AI's output. In regulated work this is not debt — it is a blocker.

02



Reference Architectures for AI

Microsoft's AI reference architectures · Azure Well-Architected for AI · OpenAI's agent patterns. Three sources of borrowed judgement — and how to use them without cargo-culting.

THE MAP

Three reference sources — what each one is actually for



Azure Well-Architected Framework — AI workloads

A set of quality tenets and decision points, organised by five pillars, with AI-specific design areas and a self-assessment.

Use as a QUESTION BANK and a review gate — it tells you what you forgot, not what to build.



Microsoft baseline reference architectures

Concrete, named, deployable blueprints — e.g. the baseline Foundry chat architecture, and its landing-zone variant with private networking and firewall.

Use as a STARTING TOPOLOGY — then document every deliberate departure as an ADR.



OpenAI's agent patterns

Vendor-neutral orchestration guidance: single agent with tools first; manager (agents-as-tools) or decentralised handoffs only when evidence demands; layered guardrails throughout.

Use as a COMPLEXITY BRAKE — it mostly tells you to build less than you planned.

Read them in this order. Patterns first (how little can I build?), then topology (what does the vendor's baseline look like?), then the framework's questions (what have I not thought about?). Reversing the order produces an over-engineered system that passes a checklist.

REFERENCE · 1 OF 3

Azure Well-Architected Framework for AI — the five pillars as a probing tool

Why it exists: AI workloads swap deterministic behaviour for non-deterministic behaviour. The WAF's AI guidance adapts the five pillars to that reality — and gives you an assessment you can run WITH the client rather than at them.

	Reliability	Model decay, fallback when inference fails, versioning and retraining paths.	“What happens when the model is wrong — not if?”
	Security	Sensitive data in prompts and training sets, access control, content safety.	“What data reaches the model, and who may see the output?”
	Cost Optimisation	Maximise value of investment, not minimise spend. Build-vs-buy, licensing, model choice.	“What is the cost per successful outcome — not per token?”
	Operational Excellence	Automation and monitoring first. CI/CD for models, drift detection, feedback loops.	“Who notices when quality drops, and how fast?”
	Performance Efficiency	Measure against an acceptable quality bar; detect degradation and decay over time.	“What is the quality bar, in a number, agreed by whom?”

How an FDE actually uses this: the right-hand column. Do not present the framework — **ask its questions**. Five questions, five silences, five constraints you did not have an hour ago.

REFERENCE · 2 OF 3

Microsoft's baseline AI architecture — the layers you will be asked to draw

The generic pattern: Microsoft's AI architecture-pattern guidance describes an intelligent-application workload built from layered components, plus an optional training/fine-tuning workload you only add if you are customising models.

Client layer

The UI or calling process. Keep it thin — delegate to the layers below.

Intelligence layer

Routing, orchestration, agents. Model routing, conversation management, decisions.

Inferencing layer

Running the model to produce predictions or generations.

Knowledge layer

The grounding data — indexed, permissioned, kept fresh by a data pipeline.

Data pipeline

Cleans, enriches and indexes source content so retrieval returns trusted, current context.



What the enterprise baselines add

- Private networking and private endpoints — no public exposure of the model endpoint.
- Zone redundancy for availability; firewall egress control.
- The landing-zone variant places all of it inside pre-governed networking, identity and policy.
- An AI gateway when you use multiple providers: one API surface, failover, chargeback per team.

HIGHLIGHT — the baseline is a starting point, not a destination. Microsoft's own framing is: start from the pattern, then adapt it to your business goals, technical constraints and risk posture. The FDE discipline is that every adaptation becomes an ADR — “we departed from the baseline here, because this constraint, accepting this consequence.” That sentence is what makes an architecture defensible in a review.

REFERENCE · 3 OF 3

OpenAI's agent patterns — the complexity brake

The guidance in one sentence: start with a single agent and add tools; split into multiple agents only when evaluations show the single agent failing. Starting multi-agent adds complexity you have not yet earned.



1 · Single agent + tools

One model, clear instructions, a growing tool set, running in a loop until an exit condition — a final-output tool, a structured output, an error, or a max-turn limit.

When Default. Prototypes and most enterprise use cases. Easiest to evaluate and debug.



2 · Manager (agents as tools)

A central manager LLM delegates to specialist agents exposed as tools, keeps control of the conversation, and synthesises the results into one answer.

When When one agent must own the user interaction and central control — e.g. a regulated workflow where one accountable path is required.



3 · Decentralised (handoffs)

Peer agents transfer control and state to one another. A triage agent hands the whole conversation to a specialist, which takes over fully.

When Triage and specialisation, where no central synthesis is needed and each specialist can own the user.

When to split — the published signals: prompts full of conditional branches that no longer scale; and tool overload — which is about tools that OVERLAP, not tool count. Some systems run 15+ distinct tools fine; others break at fewer than 10 similar ones.

Guardrails are layered, not a feature: relevance classifiers, safety filters, PII protection, moderation, tool-risk assessment, and human-in-the-loop for high-risk actions or repeated failure. In regulated work these are the architecture — not an add-on.

USING THEM WELL

How reference architectures go wrong — and the three questions that save you

Cargo-culting



Copying the diagram because it is official. The baseline assumed a context you do not have — and you inherited its costs without its reasons.

Checklist theatre



Passing every WAF question on paper while the real risk — no ground truth, no owner — is not on the checklist.

Vendor gravity



The reference architecture is published by someone who sells the components. Its default answer to “build or buy?” is not neutral.

Three questions to ask of any reference architecture

01 What context did this assume?

Scale, regulation, team size, data residency. If your context differs on any of them, the diagram is a hypothesis, not an answer.

02 Which parts are load-bearing?

Some boxes exist for the pattern to work; others are the vendor's preferred product. Separate the structure from the shopping list.

03 What would I have to believe?

For this to be right here — what must be true? Write those beliefs down; they are your assumptions register and your spike list.

The defensible sentence: “We started from <named baseline>. We kept the intelligence and knowledge layers as published. We departed on <X> because <constraint>, accepting <consequence>. We would revisit if <trigger>.” If you can say that, no architecture review can corner you.

03



Solution Decoding

From a loose business problem to a defensible architecture in ninety minutes — and the canvas that holds the result.

SOLUTION DECODING

The 90-minute decode — a clock, not a conversation

Why a time-box at all? Because unbounded discovery is how projects die politely. Ninety minutes forces the disagreement into the room while it is still cheap — and produces an artefact people can argue with.

0–10	Restate the brief back	Say it in their words, then ask “what did I get wrong?” The corrections are the real brief.
10–30	Probe for hidden constraints	The 7 questions. Do not solve. Do not agree yet. Write everything on the wall.
30–45	Name the disagreement	Make the conflict explicit and neutral: “I’m hearing two different problems. Which one pays?”
45–60	Anchor to a reference pattern	Pick a baseline out loud. State the departures and the constraint driving each.
60–75	Fill the canvas, live	On the screen, in front of them. Gaps stay visibly blank — blanks become spikes.
75–85	Size it and state confidence	T-shirt size + the cone. Never a single number without a confidence band.
85–90	The 85–90 move is the one people skip and the one that works. Restating the brief badly on purpose — slightly too narrow — makes stakeholders correct you. Corrections are free requirements, and they arrive without anyone having to admit they disagree.	

04



The FDE Solution Canvas

Problem · stakeholders · constraints · success metrics · architecture · risks · ROI. One page, filled in front of the client, with the blanks left visible.

THE ARTEFACT

The FDE Solution Canvas — seven boxes, one page, no hiding



1 • PROBLEM

One sentence, in the client's language, that a sceptic would accept. If it needs a paragraph, you have two problems — split them.

"If this works, what stops happening?"



2 • STAKEHOLDERS

Who pays, who uses, who can veto, who is accountable to the regulator. Names, not departments.

"Who can stop this in month three?"



3 • CONSTRAINTS

Regulatory, data residency, latency, budget, skills, existing contracts. The ones nobody volunteers.

"What may this system never do?"



4 • SUCCESS METRICS

A number, a baseline, a date, and a named owner of the measurement. No baseline = no success.

"What is it today, measured how?"



5 • ARCHITECTURE

The anchoring reference pattern plus every deliberate departure, each with its ADR sentence.

"What did we depart from, and why?"



6 • RISKS

Each risk gets an owner and a trigger. Untested assumptions live here until a spike kills them.

"What would I have to believe?"



7 • ROI

Value per successful outcome vs cost to run and maintain. Honest, not heroic — including the humans still in the loop.

"Who signs the business case?"

Three rules of the canvas

- Fill it live, on the screen. Not afterwards, in a document nobody reads.
- Leave blanks blank. A visible gap is a spike; a guess is a lie with a font.
- Every box needs a name. Unowned boxes are decoration.

05



Estimating Effort & Confidence

T-shirt sizing · uncertainty cones · spike planning. An estimate without a stated confidence is not an estimate — it is a hostage.

ESTIMATING · 1 OF 2

T-shirt sizing and the uncertainty cone — say the range, name the date it narrows

T-shirt sizing — coarse on purpose

S

Known pattern, known systems, one integration, ground truth exists.

M

Known pattern, one unknown — a new integration OR a new data source, not both.

L

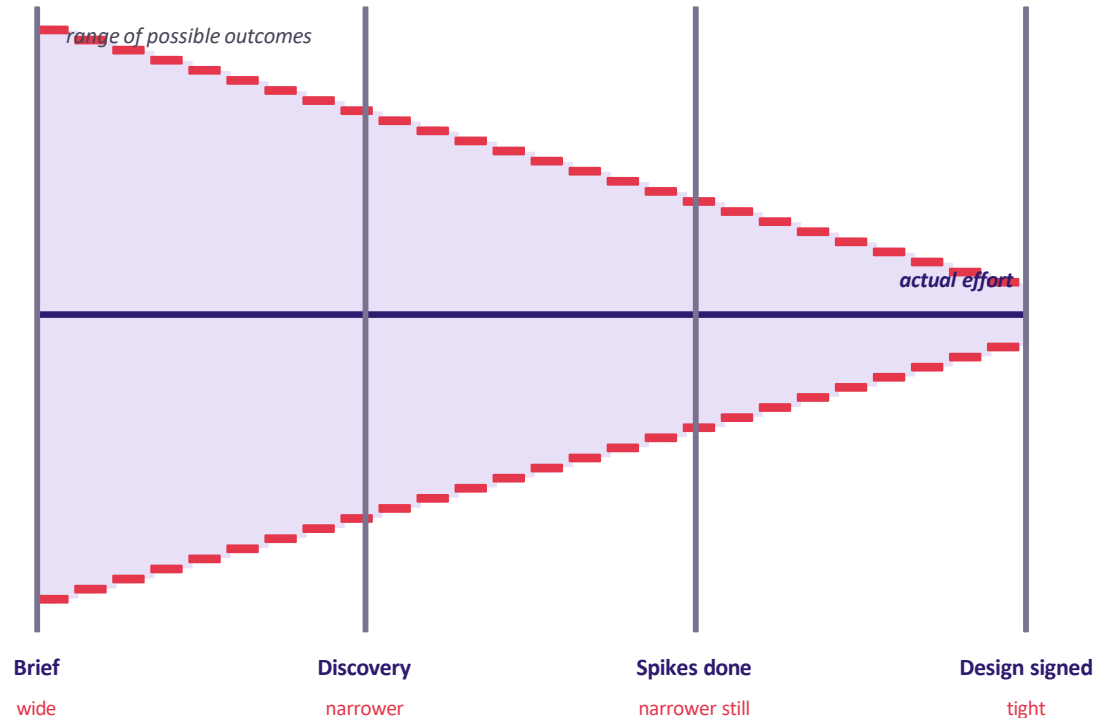
Multiple unknowns, or a regulated path, or ground truth must be built first.

XL

Unknown unknowns. Do NOT size this. Sell a spike, then size what you learn.

Why letters, not days: a number invites negotiation on the number. A size invites conversation about the unknowns — which is the conversation you actually need.

The uncertainty cone — the estimate narrows as facts arrive



Say it like this: “Today this is an L, roughly 3–6 months. After the two spikes, in three weeks, I will give you the same estimate with half the range. The range is the honest part.”

ESTIMATING · 2 OF 2

Spike planning — buy the knowledge that collapses the range

A spike is not a prototype. Its deliverable is an ANSWER, not a feature. It has a question, a time-box, a decision it unblocks, and permission to be thrown away — that last part is what makes it fast.



Question

One, falsifiable. “Can we extract structured adverse-event fields from real case emails at ≥X% agreement with a human coder?”



Time-box

Days, not sprints. When the box ends, you report what you know — including “we still don’t know, and here’s why”.



Decision unblocked

Name it. If no decision changes on the answer, it is curiosity — cut it from the plan.



Disposable

Explicitly throwaway code. The moment a spike is allowed to become the product, it stops being fast and starts being debt.

Which spikes to buy first — rank by range collapsed per day spent

Feasibility	Can the model actually do the task on YOUR data, at YOUR quality bar?	<i>Collapses the biggest range — but only if ground truth exists. If not, the ground-truth spike comes first.</i>
Access	Can we get the data, legally and technically, in the timeframe?	<i>Cheapest to run, most likely to kill the project. Run it in week one, always.</i>
Sell the spike, not the certainty. “Two weeks, two questions, and you get a halved estimate and the right to cancel.” Almost no client refuses that offer — and it converts your uncertainty from a weakness into a priced, bounded product.		

06



Stakeholder Probing

A 7-question framework to surface the constraints nobody volunteers — because the constraint that kills you is never in the brief.

THE FRAMEWORK

Seven questions that surface hidden constraints

01



“If this works perfectly, what stops happening?”

Forces a concrete outcome and exposes whether they want efficiency, coverage, or cover.

02



“Who has tried this before here, and what happened?”

There is always a previous attempt. Its corpse holds the real constraint — and the political scar tissue.

03



“What may this system never do?”

The negative-space question. Answers arrive as regulation, union agreements, and promises made to auditors.

04



“Who signs off, and who can veto without signing?”

Separates the sponsor from the blocker. The blocker is usually absent from the kick-off — invite them.

05



“What is the number today, and who measures it?”

No baseline, no success. If nobody measures it, your first deliverable is measurement — say so now.

06



“What happens when it is wrong?”

Not IF. Establishes tolerance, the human in the loop, and the liability path in one question.

07



“What would make you cancel this in month three?”

The best question in the set. People will name their true risk when invited to imagine failure.

Technique, not script: ask, then stop talking. The constraint lives in the silence after the question — and in the glance between two stakeholders before either answers.

07



Case Study — “Add AI to Pharmacovigilance”

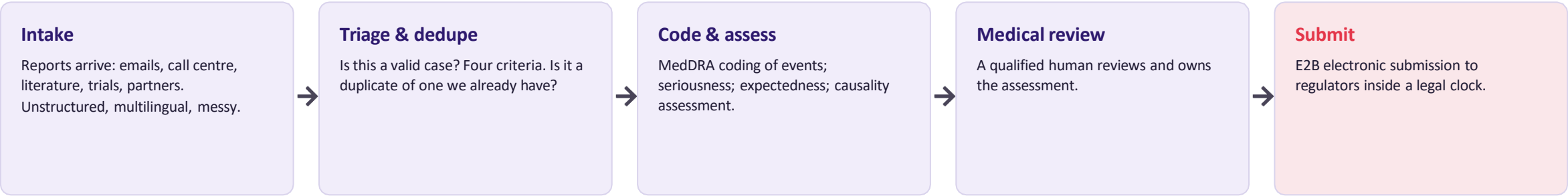
Five words. Three stakeholders who mean three different projects. One regulator watching. Ninety minutes to a scope you can defend.

DOMAIN PRIMER

Pharmacovigilance in five minutes — enough to decode it, not to practise it

Pharmacovigilance (PV) is drug safety monitoring: detecting, assessing, understanding and preventing adverse effects of medicines once they are on the market. It is a legal obligation, not a business choice — and the regulator can inspect it.

The case-processing pipeline — where every AI idea will land



The words your stakeholders will use

ICSR	Individual Case Safety Report — one adverse-event report about one patient. The atomic unit of PV.	MedDRA	The standard medical terminology used to code events. Hierarchical; coding choices change what a case means.
Seriousness	A regulatory classification (death, hospitalisation, etc.) — not a clinical opinion. It sets the legal clock.	Expedited reporting	Serious + unexpected cases must reach the regulator inside a short legal window. Missing it is an inspection finding.
Signal detection	Statistical scanning of the safety database for previously unknown risks. A separate discipline from case processing.	QPPV	Qualified Person for Pharmacovigilance — a named individual personally accountable to the regulator for the PV system.

The one fact that decides this whole case study: PV has been processing and coding cases by hand for decades. That history is a labelled dataset. **Ground truth already exists.**

MINUTE 0

The brief — and the three projects hiding inside it

THE BRIEF: *"We need to add AI to pharmacovigilance. Everyone's doing it. Budget's approved — come back with a plan."*



Head of PV Operations

"Case volume is up 40% and headcount is flat. I need the processing cost per case down."

Means: Automate intake, dedupe and coding.

Real fear: Her team drowning — and an inspection finding for a late case.



Head of Safety Science

"I don't care about typing. I want to find signals earlier than the regulator does."

Means: AI on signal detection over the safety database.

Real fear: Being told about a safety issue by the EMA rather than telling them.



QPPV / PV Compliance

"I am personally named to the regulator. Nothing goes near the submission path."

Means: Ideally, nothing at all. Realistically: nothing autonomous.

Real fear: Signing off a system she cannot explain in an inspection.

The decode begins here. These are not three opinions about one project — they are three different projects with different regulatory exposure, different data, and different ground truth. The junior move is to average them into a platform that serves nobody. The FDE move is to say out loud: *"I'm hearing three projects. Let's pick the one that pays and name the other two as later."*

MINUTES 10–60

The decode — the probing questions that collapsed the scope

Q01

“If this works perfectly, what stops happening?”

Ops: “My team stops re-typing the same case from three sources.” Science: “We stop finding out from the regulator.”

→ Two different 'perfect'. Confirms two projects.

Q03

“What may this system never do?”

QPPV, instantly: “It may never decide seriousness on its own, and it may never submit anything.”

→ The single most valuable sentence of the day. That is the architecture.

Q05

“What is the number today, and who measures it?”

Ops has it: cases/FTE/month, cycle time, and QC error rate — all measured, all trended, for years.

→ A real baseline exists. Success is provable. Rare — bank it.

Q02

“Who tried this before, and what happened?”

A vendor auto-coding tool was piloted two years ago; QA rejected it — no audit trail on why it chose a code.

→ The constraint is explainability and audit, not accuracy.

Q06

“What happens when it is wrong?”

A miscoded serious case = a missed expedited submission = an inspection finding. A missed dedupe = a corrupted signal.

→ Asymmetric error cost. False negatives are far worse than false positives.

Q07

“What would make you cancel this in month three?”

QPPV: “If I can't explain it to an inspector in one page, I'll stop it — whatever it saves.”

→ The acceptance criterion is a document, not a metric.

Fifty minutes, six questions, zero architecture drawn — and the solution is now almost forced. That is what decoding is: removing the wrong solutions until the right one is the cheapest thing left standing.

MINUTES 60–75

The canvas, filled live — blanks left visible

PROBLEM PV case intake cannot scale with volume: the same case arrives from multiple sources and is re-keyed and re-coded by hand, consuming reviewer time that should go to medical judgement.	STAKEHOLDERS Pays: Head of PV Ops. Uses: case processors + medical reviewers. Vetoes: QPPV (accountable to regulator). Deferred: Head of Safety Science — signal detection is phase 2.	CONSTRAINTS AI may never determine seriousness alone, nor submit to a regulator. Explainable to an inspector in one page. Patient data stays in-region. Model changes are controlled changes, not deploys.	SUCCESS METRICS Baseline exists: cases/FTE/month, cycle time, QC error rate. Target: cycle-time reduction with QC error rate NOT worse, agreement with human coders measured by Cohen's Kappa on held-out historical cases.
ARCHITECTURE Single agent + tools (per OpenAI guidance) on the enterprise baseline topology: private networking, in-region inference, human-decision gate. Departure from baseline: model version pinned, no auto-update.	RISKS Asymmetric error cost — false negatives on serious cases dominate. Explainability accepted by QA (not yet proven → SPIKE). MedDRA licence terms for AI use (BLANK — unknown, owner: Legal, due Friday).	ROI Reviewer hours returned to medical judgement, not headcount removed. Humans stay in the loop by design — the business case must say so, or it is not defensible.	 The blank is the point The MedDRA licensing question had no answer in the room. It stayed blank on the screen, with a name and a date next to it. That blank bought more credibility than any of the filled boxes. Clients have met confident consultants before. They have rarely met one who admits a gap and assigns it.

MINUTE 90

The defensible scope — and the NOT list read out loud



IN SCOPE — phase 1, twelve weeks

- Intake triage: is this a valid case? Extract the four criteria from unstructured text — AI proposes, human confirms.
- Duplicate detection across sources — the highest-volume, lowest-regulatory-risk win.
- MedDRA coding SUGGESTIONS with confidence and a cited source span. The coder accepts or overrides; both are logged.
- Full audit trail: input, suggestion, confidence, model version, human decision, timestamp.
- Eval harness against historical human-coded cases; agreement by Cohen's Kappa; recall-weighted on serious cases.



NOT IN SCOPE — said out loud, and agreed

- Autonomous seriousness determination. The human decides. Non-negotiable — the QPPV's own words.
- Anything that writes to, or triggers, the regulatory submission path.
- Signal detection — a different discipline, different regulatory exposure, sparse ground truth. Phase 2, after phase 1 earns trust.
- Auto-updating models. Version pinned; an update is a controlled change with re-validation.
- Headcount reduction. Not promised, not modelled, not implied.

Why this survives an inspection — the regulator-aware moves

Human-centric by design

The EMA's reflection paper puts a human-centric, risk-based approach at the core. Our human decides every case; the AI never does.

Risk sits where impact sits

Regulatory impact scales with consequence. We took the low-impact, high-volume work and left the high-impact judgement alone.

Interpretability + oversight

Where full explainability is impossible, the expectation shifts to interpretability with demonstrable human oversight — hence cited source spans and a logged human decision.

Monitored, not installed

AI touching AE classification must be closely monitored by the marketing authorisation holder — so monitoring is in phase 1, not a phase 3 nice-to-have.

WRAP-UP

What you can do tomorrow that you could not do yesterday



Shipped this session

- A discovery method that reads running evidence — pipelines, billing, on-call — instead of trusting the wall diagram.
- Three reference sources and the division of labour between them: patterns brake complexity, baselines give topology, the framework supplies the questions.
- The 90-minute decode clock, ending in a NOT list read aloud.
- The FDE Solution Canvas — seven boxes, blanks left visible.
- Estimates with a stated confidence, and spikes that collapse the range.
- Seven questions that surface what nobody volunteers.



Your assignment before next session

- Take a live brief from your own account — one sentence, real ambiguity.
- Run the 7 questions against a colleague playing your hardest stakeholder. Record what you did NOT get on the first pass.
- Fill the canvas. Leave the blanks blank; assign each one a name and a date.
- Size it, state the confidence band, and name the two spikes that would halve it.
- Bring the NOT list. We will attack it — the same way your architect mentor attacked ADR-3.

The through-line: “add AI to pharmacovigilance” was never a technical problem. Every hard constraint that shaped the final architecture came out of a stakeholder's mouth in the first fifty minutes — and would have surfaced anyway, in month four, as a cancellation. The Forward Deployed Engineer's craft is *pulling the future objection into the present*, while it is still free to act on. Discovery is not the work before the work. It *is* the work.

Bring your worst brief. We'll decode it together.

One real sentence from a real account. The 7 questions. The canvas, with blanks. A size, a confidence band, and two spikes. And a NOT list you are willing to defend.

Discover

Anchor

Decode

Commit